COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 17 Operating systems and resource management

Individual module / Expanded Volume 04

Four lessons with original foundations, expanded study, worked examples, laboratory checkpoints and quizzes.

Lessons 17.1–17.4 · 20 questions · One integration project

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Module 17: Operating systems and resource management

Original prerequisites: Module 16.

Integration goal: Operating-system investigation. Use only ordinary user privileges and disposable files. Compare execution and storage mechanisms while keeping the contents and useful work constant.

17.1 — Processes, threads, system calls and protection

Learning objective: Explain which resources are isolated and which are shared.

Original foundation

A process is an execution environment with an address space and operating-system-managed resources. Threads within a process share memory but have their own execution state. A system call requests a kernel service through a controlled boundary. User/kernel separation protects privileged operations; it does not make application code automatically safe. Process isolation helps contain faults, while threads allow lower-overhead sharing at the cost of synchronization complexity. Starting a worker process, a thread and an asynchronous task are not interchangeable choices. Identify ownership, failure propagation and communication needs before choosing a mechanism.

Expanded study

Make an ownership map first

A thread has its own execution state but can reach the process's shared objects. A separate process normally needs an explicit exchange to transfer information. Neither choice automatically answers who owns an input buffer, who reports an exception, or who waits for completion. Draw the worker's input, output and lifetime before selecting the mechanism. A future is a handle to a result, not a new protection boundary by itself. [R16, P08, P14]

Use a process boundary for a narrow experiment

The reference launches a fixed Python child program with an argument list and no shell. The parent validates at most 256 small integers, serializes them to standard input, and receives a result plus the child's process ID. The child has a fresh interpreter and reconstructs its own list from bytes. Changing that list would not directly mutate the parent's original list. Shared-memory APIs could change the design, but are not used here. [P08]

Failure belongs in the interface

The parent sets a timeout, checks the exit status and parses the response. A nonzero exit is not a valid numerical result; malformed output is not success just because the process ended. A timeout requires cleanup and a clear error path. If a worker can perform durable external effects, termination alone cannot undo them. The fixed child in this lab only computes and prints, making the boundary small enough to reason about. [P08]

Do not mistake isolation for a sandbox

A child typically starts with access derived from the parent account. It can still read or write whatever its environment permits unless additional controls are imposed. The exercises never launch untrusted source code or request-supplied executable paths. Use a private scratch directory and ordinary user permissions. Operating-system isolation is useful fault containment, but the teaching subprocess wrapper is not a security boundary for hostile tools.

Worked reasoning

The parent sends [2,3,5]. The child computes 10 and reports a process ID different from the parent's. The supplied demo deliberately prints only whether the identities differ, because actual numeric IDs vary between runs. Compare that with a C++ thread capturing a reference to one Inventory object: the thread can reach the same object, so its updates require the locking contract introduced in Module 18.

Runnable example

File: `python/examples/lesson_17_1.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.1 --build-dir build
```

```
import os
from python.reference.lab_17_1 import run_child
r=run_child([2,3,5])
print("sum=",r["sum"]," separate_process=",r["pid"] != os.getpid(),sep="")
```

Expected output from the supplied reference in the validated environment:

```
sum=10 separate_process=True
```

Practical checkpoint

Write a program that reports its process identity and runs independent worker tasks. Pass: explain what memory is shared in the chosen approach and how worker failure is reported without privileged operations.

Expanded contract: Python run_child(list[int],timeout=3.0) accepts at most 256 integers in 0..1000 and timeout in (0,30]. It runs only a fixed child program, returns integer pid/sum fields, and preserves the input. Errors propagate; no shell is invoked.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_1.py`. Use `python3 check_lab.py 17.1 --student` after implementation.

Independent transfer

A child process can still overwrite a file owned by the same account. Does that contradict process memory isolation?

Quiz — answer before opening the key

1. **What do threads in one process typically share?**
2. **What is a system call?**
3. **Why might processes be preferable to threads for a risky tool?**
4. **Why is an argument list with shell=False useful here?**
5. **What is wrong with returning zero whenever a worker crashes?**

Assigned study

R16: Processes, Process API and Threads.

Expansion source map: R16, P08, P14. See the source directory for the exact documents and version notes.

17.2 — Allocation, paging and memory-mapped data

Learning objective: Distinguish reserved address space, committed storage and live objects.

Original foundation

An allocator manages storage for application objects; the operating system manages virtual memory and backing resources. Reserving an address range is not the same as having every page physically resident. Page faults can occur when access requires additional mapping or data movement. Application allocation behavior and operating-system paging interact but are not identical. Memory mapping can present file data through an address range, but it still requires lifetime, bounds and error handling. It does not guarantee zero I/O latency. Avoid assuming that freeing an object immediately reduces the process's visible memory usage: an allocator may retain storage for reuse.

Expanded study

Track four different quantities

An object count, allocator reservation, virtual-address range and resident physical storage are different measurements. Destroying an object does not necessarily release its containing allocation, and releasing an allocation does not necessarily cause the runtime to return pages immediately. A memory-mapped file gives an addressable view of file bytes; it does not create a collection of C++ objects merely by assigning a pointer type. [R16, P07]

Read-only mapping has a lifetime

The reference opens a small regular file, maps it read-only, computes a digest while the mapping is live, then closes the mapping before leaving the file context. No view escapes to a later task. An empty file follows a special path because mapping zero bytes is not a portable way to handle emptiness. The file must remain unchanged during the comparison; a concurrent truncation would violate the lab's environmental assumption. [P07]

Make comparisons about identical content

The streaming path hashes fixed-size chunks. The mapping path passes the mapping to the same digest operation without converting the entire file to a new bytes object. Equal digests are a practical content check for this experiment, and direct small-fixture comparisons can supplement them. This does not establish that either access path performs fewer storage operations: cache warmth, page faults and buffering affect both.

Design a trustworthy timing extension

Create the input outside the timed region and state whether opening, mapping, hashing and closing are included. Change test order across trials. Do not call a second run a cold-cache measurement merely because it uses a different API. This course does not ask the learner to flush system caches, change kernel settings or allocate enough memory to destabilize the workstation. Keep the baseline small and report what was actually controlled.

Worked reasoning

A page-aligned 4100-byte region spans two chosen 4096-byte pages, although only four bytes lie in the second page. That is page-coverage arithmetic, not a resident-set measurement. The demo writes 4096 known bytes and obtains identical streamed/mapped digests. Its useful conclusion is content agreement under the no-concurrent-writer assumption, not a claim that mapping is universally faster.

Runnable example

File: `python/examples/lesson_17_2.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.2 --build-dir build
```

```
import tempfile
from python.reference.lab_17_2 import file_digests
with tempfile.TemporaryDirectory() as d:
    path=Path(d)/"asset.bin"
    path.write_bytes(bytes(range(256))*16)
    a,b=file_digests(path)
    print("equal=",a==b," bytes=",path.stat().st_size,sep="")
```

Expected output from the supplied reference in the validated environment:

```
equal=True bytes=4096
```

Practical checkpoint

Compare sequential file reads with a read-only mapped-file approach on a small disposable file. Pass: verify identical content and report virtual/resident concepts separately without promising one method is faster.

Expanded contract: Python file_digests(path) returns two SHA-256 hex digests for a regular file no larger than 1 MiB. The file resides in a trusted scratch directory and is not modified concurrently. Empty files are supported; no view escapes the mapping context.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_2.py`. Use `python3 check_lab.py 17.2 --student` after implementation.

Independent transfer

An allocator frees 1000 game entities but the process memory graph stays flat. Is that evidence of a leak by itself?

Quiz — answer before opening the key

1. **Does reserving virtual address space imply all pages are resident?**
2. **Why may process memory usage stay high after freeing objects?**
3. **Does memory mapping eliminate storage latency?**
4. **Why does the empty-file case need an explicit branch?**
5. **Why should the mapping be closed before its resources are discarded?**

Assigned study

R16: Address Spaces, Address Translation, Paging and Memory API.

Expansion source map: R16, P07. See the source directory for the exact documents and version notes.

17.3 — Scheduling, responsiveness and real-time constraints

Learning objective: Distinguish average completion from meeting a deadline.

Original foundation

A scheduler chooses runnable work according to its policy. Context switches have costs, and oversubscription can create contention rather than useful parallelism. Throughput-oriented scheduling is not necessarily suitable for a time-critical audio or input path. Real-time reasoning concerns deadlines and worst-case behavior under stated assumptions. General desktop operating systems do not make arbitrary user programs hard-real-time merely because a thread has high priority. Priority inversion occurs when high-priority work waits indirectly on lower-priority work holding a needed resource. Design short critical sections and bounded work before experimenting with priorities.

Expanded study

Define the deadline in the same clock domain

A deadline is a time by which a required result must be ready, not just a hoped-for average duration. The scheduler model uses release time, execution duration and an absolute deadline, all in the same invented time units. Start time cannot precede release. Finish equals start plus duration. Positive lateness is max(0, finish-deadline). This isolates scheduling arithmetic before studying noisy real measurements.

Blocking can dominate a lightly loaded machine

The model is nonpreemptive first-come, first-served. Once a long background job starts, a newly released audio-like job must wait. Average utilization over a long interval might be low while that one wait misses a short deadline. The remedy may involve shorter work units, an appropriate priority policy or eliminating shared blocking resources. Adding workers is not a guaranteed remedy when the bottleneck is one serialized resource. [R16]

Distinguish inversion from ordinary contention

Priority inversion specifically concerns a higher-priority task waiting on a resource held by lower-priority work, potentially while medium-priority work delays the holder. The FCFS example demonstrates blocking latency, not a complete priority-inversion implementation. Explain the three-task resource timeline on paper rather than assigning privileged thread priorities on a primary machine. A standard desktop process is not made hard-real-time by a thread label. [R16]

Choose a structural repair

An audio callback should not perform file loading under a mutex needed by a background importer. One repair is to prepare immutable data elsewhere and publish a ready handle through a bounded handoff. The callback then performs a small predictable operation or uses a fallback if data is unavailable. Such a design still requires a correct ownership protocol; removing a mutex without replacing its lifetime guarantee creates a different failure.

Worked reasoning

Background job B is released at 0, lasts 12 units and has deadline 100. Audio-like job A is released at 1, lasts 1 and has deadline 5. B runs 0..12; A runs 12..13 and is eight units late. A revised admission plan may release a two-unit preparation job at 0, A at 1, and the remaining background work at 3. A then completes at 3. This is a changed model and workload admission policy, not proof that any OS will reproduce the schedule.

Runnable example

File: `python/examples/lesson_17_3.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.3 --build-dir build
```

```
from python.reference.lab_17_3 import schedule_fcfs
for row in schedule_fcfs([(0,12,100),(1,1,5)]):
    print(row)
print("Times above are model units, not measured OS latency.")
```

Expected output from the supplied reference in the validated environment:

```
{'job': 0, 'start': 0, 'finish': 12, 'lateness': 0}
{'job': 1, 'start': 12, 'finish': 13, 'lateness': 8}
Times above are model units, not measured OS latency.
```

Practical checkpoint

Simulate a scheduler or instrument a bounded producer/consumer workload without changing privileged priorities. Pass: show one contention-induced latency spike and propose a structural fix rather than only raising priority.

Expanded contract: Python schedule_fcfs accepts at most 128 jobs (release,duration,absolute_deadline), with bounded nonnegative integer times, positive duration and deadline >= release. It is nonpreemptive, ties use input order, and outputs include start, finish and nonnegative lateness.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_3.py`. Use `python3 check_lab.py 17.3 --student` after implementation.

Independent transfer

A performance report says average frame cost is 4 ms, so every 8 ms deadline must be met. Find the missing evidence.

Quiz — answer before opening the key

1. **What is priority inversion?**
2. **Does low average CPU use guarantee deadlines are met?**
3. **Why avoid blocking I/O in an audio callback?**
4. **A job released at 4 starts at 9, runs for 2 and has absolute deadline 10. What are waiting time and lateness?**
5. **Why is splitting a background task insufficient if all chunks still monopolize a shared lock?**

Assigned study

R16: Scheduling, Multi-Level Feedback Queue and Concurrency Bugs.

Expansion source map: R16. See the source directory for the exact documents and version notes.

17.4 — Filesystems, persistence and asynchronous I/O

Learning objective: Design data operations that survive partial failure.

Original foundation

A filesystem organizes persistent data and metadata. A successful write call does not always mean data has reached durable storage. Buffering, ordering and crash consistency require explicit treatment when durability matters. A robust save process avoids overwriting the only valid copy with partially generated data.

Asynchronous I/O lets work proceed without blocking the caller, but completion can occur later and fail. Buffers and request objects must remain valid until completion according to the API. Cancellation is a request with a contract, not necessarily immediate erasure of all work. Separate correctness, durability and performance goals.

Expanded study

Separate visibility from durability

A save operation has several milestones: valid data in memory, bytes written to a temporary file, buffers flushed, replacement of the destination name, and whatever additional persistence operations the platform requires. An operation returning from write is not the same event as survival of a power loss. The reference performs file flush/fsync and replacement but does not promise parent-directory synchronization or a complete storage-device failure guarantee. [R16, P06]

Preserve the last usable state before commitment

Validate the complete small save schema before touching the destination. Create a unique temporary file in the same trusted directory, write the candidate there, close it and replace the destination. A failure before replacement leaves the previous destination available. Clean up abandoned temporary files owned by this operation. The directory is assumed private and there is one writer; this is not a defense against hostile concurrent path replacement.

Treat post-commit errors differently

An exception injected after replacement occurs after the new file has become visible. Reporting failure does not make the old file reappear. Recovery must inspect the actual committed state rather than blindly retrying a non-idempotent effect. For this save format, replacing with the same validated snapshot is naturally repeatable. The learner must still explain the distinction between a rejected candidate and an uncertain response after commit.

An asynchronous request must own its inputs

The added write_save_async helper takes a private validated snapshot before placing blocking file work on an executor. The caller can change its original dictionary afterward without altering the worker's snapshot. Keep only one outstanding write per destination in this design. A future records completion or failure; it does not eliminate disk latency. This adapter moves blocking work to a thread and is not a kernel-native asynchronous I/O implementation. [P14]

Worked reasoning

Begin with score 7. Inject a failure after half the temporary payload is written: the destination still loads as score 7 and the incomplete temporary file is removed. Now write score 99 and inject after replacement: readback returns 99 even though the caller received an exception. Test both outcomes. A report that calls every exception a rollback would be contradicted by the second experiment.

Runnable example

File: `python/examples/lesson_17_4.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.4 --build-dir build
```

```
import tempfile
from python.reference.lab_17_4 import write_save,read_save,InjectedFailure
with tempfile.TemporaryDirectory() as d:
    path=Path(d)/"save.json"
    write_save(path,{"version":1,"score":7})
    try:
        write_save(path,{"version":1,"score":99},"partial_write")
    except InjectedFailure:
        print("injected failure; old score=",read_save(path)["score"],sep="")
    write_save(path,{"version":1,"score":99})
    print("new score=",read_save(path)["score"],sep="")
```

Expected output from the supplied reference in the validated environment:

```
injected failure; old score=7
new score=99
```

Practical checkpoint

Build a save pipeline with injected failures before, during and after serialization. Pass: recover a valid prior or new save under your documented model and never reuse an asynchronous buffer before completion.

Expanded contract: Python write_save/read_save use exactly version=1 and integer score in 0..1000000. write_save supports four named injection points. Readback is capped at 1024 bytes. Guarantees cover ordinary process exceptions on a trusted single-writer scratch path, not power-loss or adversarial-filesystem safety.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_4.py`. Use `python3 check_lab.py 17.4 --student` after implementation.

Independent transfer

The caller times out waiting for an asynchronous save future. May it immediately reuse a mutable buffer that the worker might still read?

Quiz — answer before opening the key

1. **Does every successful write imply durable storage?**
2. **Why retain an asynchronous I/O buffer until completion?**
3. **What should a save-recovery test inject?**
4. **What state is expected after the reference injects after_replace?**
5. **Why create the temporary file in the destination directory?**

Assigned study

R16: Files and Directories, Crash Consistency and Journaling.

Expansion source map: R16, P06, P14. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Create a systems lab report covering processes, memory, scheduling and file I/O. Use unprivileged programs and disposable data; do not alter kernel or device settings on a primary machine.

Use only ordinary user privileges and disposable files. Compare execution and storage mechanisms while keeping the contents and useful work constant.

1. A process/thread ownership table, a child-process run and a reported worker error.
2. Identical stream/mapping digests, the empty-file case, bounds and map lifetime notes.
3. A scheduler trace with an explicit missed deadline; label all synthetic times as model units.
4. Before/during/after replacement save-failure tests, including observed old/new destination state.
5. An asynchronous offload lifetime diagram and completion/error check; no power-loss or hard-real-time claim.

Live review: An asynchronous save returns an exception after replacement. Explain why retry/recovery must inspect state rather than assuming that no write occurred.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.